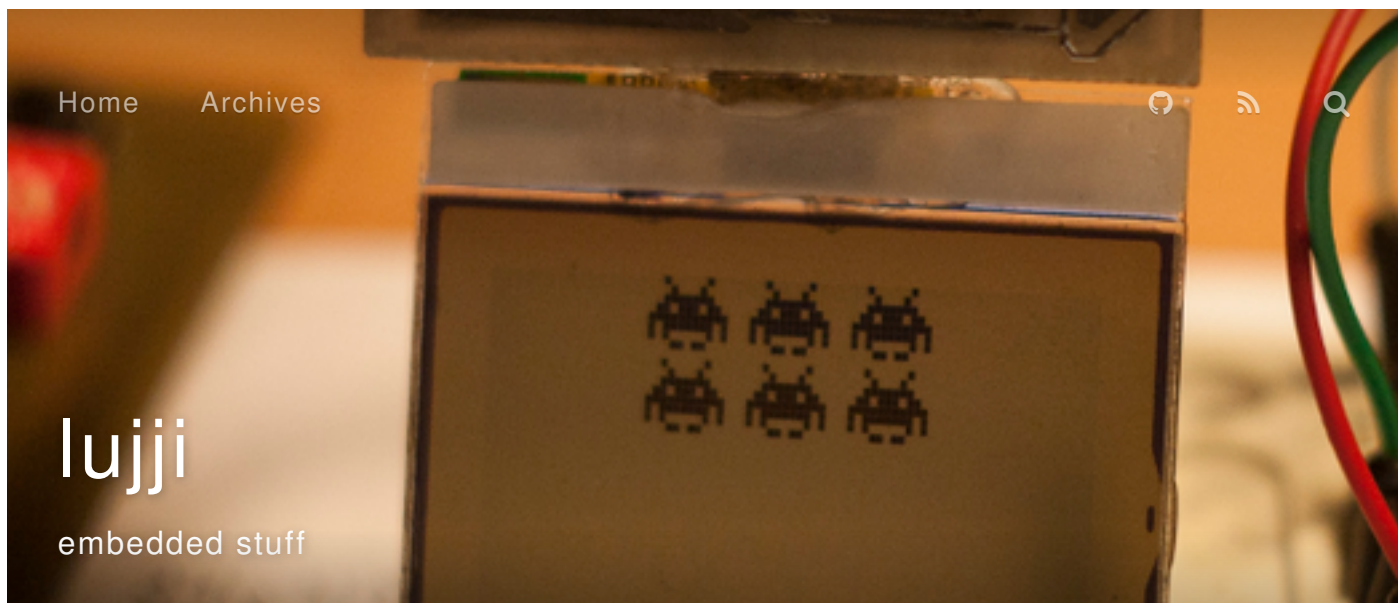




2017-03-04

Bare metal programming: STM8

This article will cover developing for STM8 series of microcontrollers completely from scratch, without using any vendor-supplied libraries.

Preface

STM8 is a cheap 8-bit microcontroller aimed towards low-cost mass-market devices. Initially I came across this part while searching for a simple microcontroller as a replacement for AVR. Despite having various ARM Cortex-M0 devices available on the market for quite attractive prices, AVR has one advantage - simplicity. Utilizing an ARM Cortex core to switch some lights on and off seems like an overkill. Some applications just don't require that amount of flexibility and performance.

The main goal of this article is to demonstrate that 'bare metal' programming is not a difficult task and to give you an overview of STM8's architecture and peripherals. Even though writing peripheral drivers from scratch might seem like reinventing the wheel, in many cases it is easier and faster to implement the functionality that *you* need for a specific task, instead of relying on vendor-supplied libraries that try to do everything at once (and fail).

Contents:

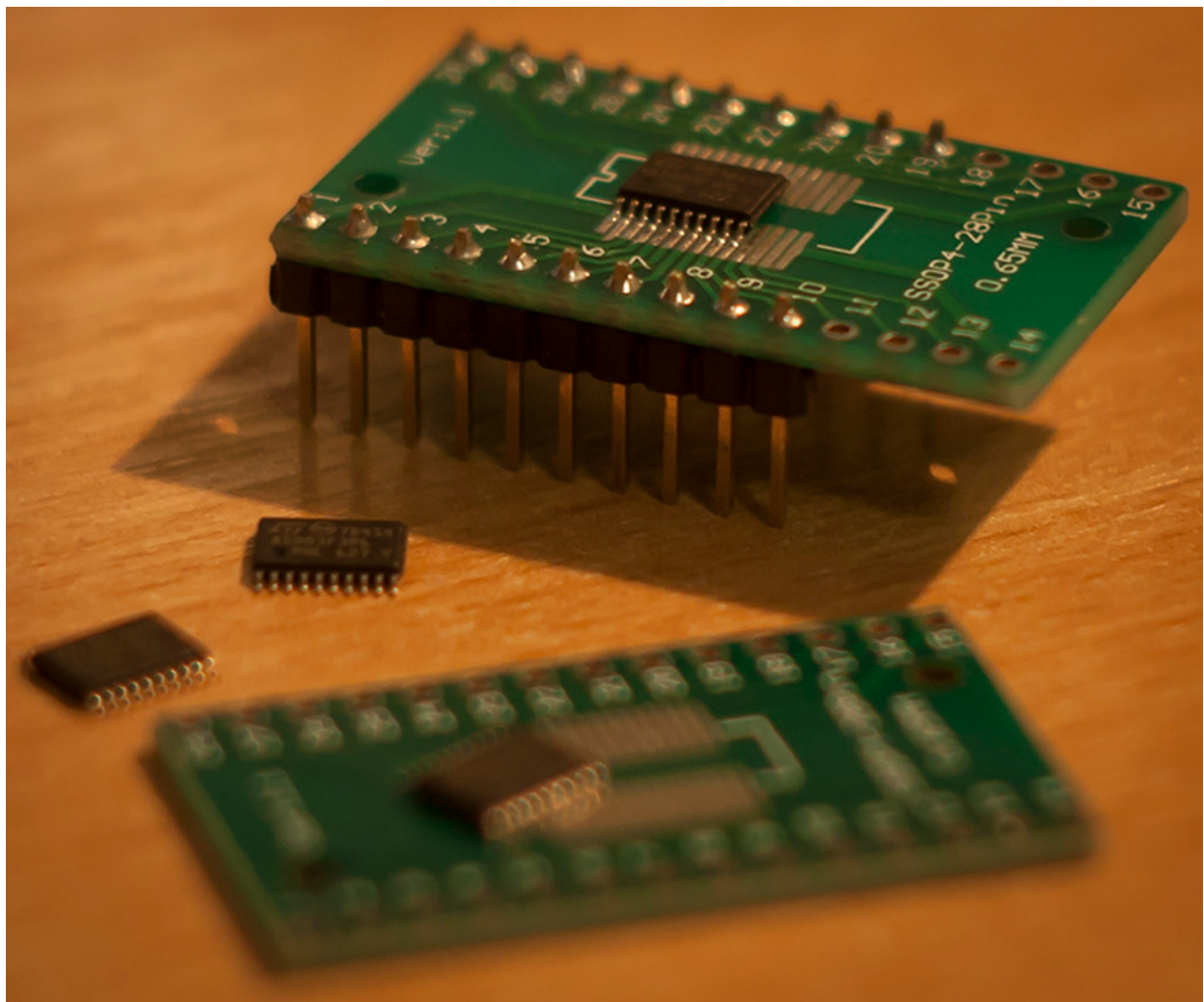
- [The Hardware](#)
- [Setting up toolchain](#)
- [It's all just memory..](#)
- [First program](#)

- [Peripheral drivers](#)
 - [UART](#)
 - [SPI](#)
 - [I2C](#)
 - [ADC](#)
 - [Timers and interrupts](#)
- [Putting it all together](#)
- [Conclusion](#)

The Hardware

There is a number of ways to start working with STM8. The easiest one is to get a Discovery board, although I wouldn't recommend it, since STM8 Discovery boards aren't that good and the on-board ST-Link v1 firmware just sucks.

Instead, I'll opt for the minimalist approach. All you need is an ST-Link v2, STM8S003F3 and a breakout board. STM8S003F3 comes in a handy TSSOP20 package which is very easy to solder.



Poor man's devboard

Note: a 1uF capacitor on VCAP pin is required for the processor to operate.

Setting up toolchain

The biggest downside is that STM8 processors are not supported by GCC. There are 3 commercial compilers available for these processors: Raisonance, Cosmic and IAR. Some of these compilers have free versions with code size limit, but none of them are available for linux. Luckily, SDCC supports STM8 and that's what we're going to use. SDCC is being actively developed, so I suggest trying the latest snapshot build instead of the stable version. To program the microcontroller we'll be using stm8flash. The first step is to download all the necessary tools:

1. [sdcc](#)
2. [stm8flash](#)

Extract SDCC under `~/local/sdcc`. Now extract stm8flash, build it with `make` and copy stm8flash binary to `~/local/sdcc/bin`. I prefer to keep flasher with compiler for convenience. Next, add the following line to your `.bashrc` file (replacing *username* with your user name):

```
1 export PATH=$PATH:/home/username/local/sdcc
```

If everything was done properly, you should be able to run `sdcc --version`. The last remaining thing is to write udev rule for ST-Link programmer. Create a file `/etc/udev/rules.d/99-stlink.rules`:

```
1 # ST-Link v1/v2
2 ATTRS{idVendor}=="0483", ATTRS{idProduct}=="3744", MODE="0666"
3 ATTRS{idVendor}=="0483", ATTRS{idProduct}=="3748", MODE="0666"
```

Finally, run `udevadm control --reload-rules && udevadm trigger` as root. Now we're all set and ready to start.

It's all just memory..

Before we begin, let's take a simple example of accessing port register on ATmega and see what's going on under the hood:

```
1 /* Port access operation */
2 PORTB = (1 << PB2);
3
4 /* Expanding macros (same as above) */
5 (* (volatile uint8_t *) ((0x05) + 0x20)) = (1 << 2);
6
7 /* Same as above */
```

```
8 * (volatile uint8_t *) 0x25 = 0x04;
```

Typecasting integer to a pointer is a valid operation in C. If you don't quite understand what is going on with pointer arithmetics then here's another example for you:

```
1 uint8_t a = 0xDE; // a contains 0xDE
2 uint8_t *ptr = &a; // ptr points to a
3 *ptr = 0xAD; // a contains 0xAD
```

The only difference is that in the first example we know exactly which address in memory we are going to use. It's important that you understand what's going on here, since we're going to use this mechanism for accessing hardware registers later on.

First program

These are the two most important documents: [datasheet](#) and [reference manual](#). We'll use the datasheet for the pinout and register map. Everything else is present in the reference manual: peripheral operation, register description, etc. Let's begin by opening the GPIO section of the reference manual and taking a closer look at PORTD registers.

0x00 500F	Port D	PD_ODR	Port D data output latch register	0x00
0x00 5010		PD_IDR	Port D input pin value register	0xFF ⁽¹⁾
0x00 5011		PD_DDR	Port D data direction register	0x00
0x00 5012		PD_CR1	Port D control register 1	0x02
0x00 5013		PD_CR2	Port D control register 2	0x00

PORTD registers

These registers are pretty much self-explanatory but just in case, here's a brief overview: **DDR** is the direction register, which configures a pin as either an input or an output. After we configured **DDR** we can use **ODR** for writing or **IDR** for reading pin state. Control registers **CR1** and **CR2** are used for configuring internal pull-ups, output speed and selecting between push-pull or pseudo open-drain.

First, let's define a macro that we'll use later on for register definitions. Base address for all the hardware registers is 0x5000 so we can hardcode that into our macro.

```
1 #define _SFR_(mem_addr) (*(volatile uint8_t *) (0x5000 + (mem_addr)))
```

Now let's try blinking an LED. For this task we need to define **ODR**, **DDR** and **CR1** registers for PORTD. We also need a delay function.

```
1  #include <stdint.h>
2
3  #define F_CPU 2000000UL
4
5  #define _SFR_(mem_addr)      (*(volatile uint8_t *) (0x5000 + (mem_addr)))
6
7  /* PORT D */
8  #define PD_ODR      _SFR_(0x0F)
9  #define PD_DDR      _SFR_(0x11)
10 #define PD_CR1      _SFR_(0x12)
11
12 #define LED_PIN      4
13
14 static inline void delay_ms(uint16_t ms) {
15     uint32_t i;
16     for (i = 0; i < ((F_CPU / 18000UL) * ms); i++)
17         __asm__("nop");
18 }
19
20 void main() {
21     PD_DDR |= (1 << LED_PIN); // configure PD4 as output
22     PD_CR1 |= (1 << LED_PIN); // push-pull mode
23
24     while (1) {
25         /* toggle pin every 250ms */
26         PD_ODR ^= (1 << LED_PIN);
27         delay_ms(250);
28     }
29 }
```

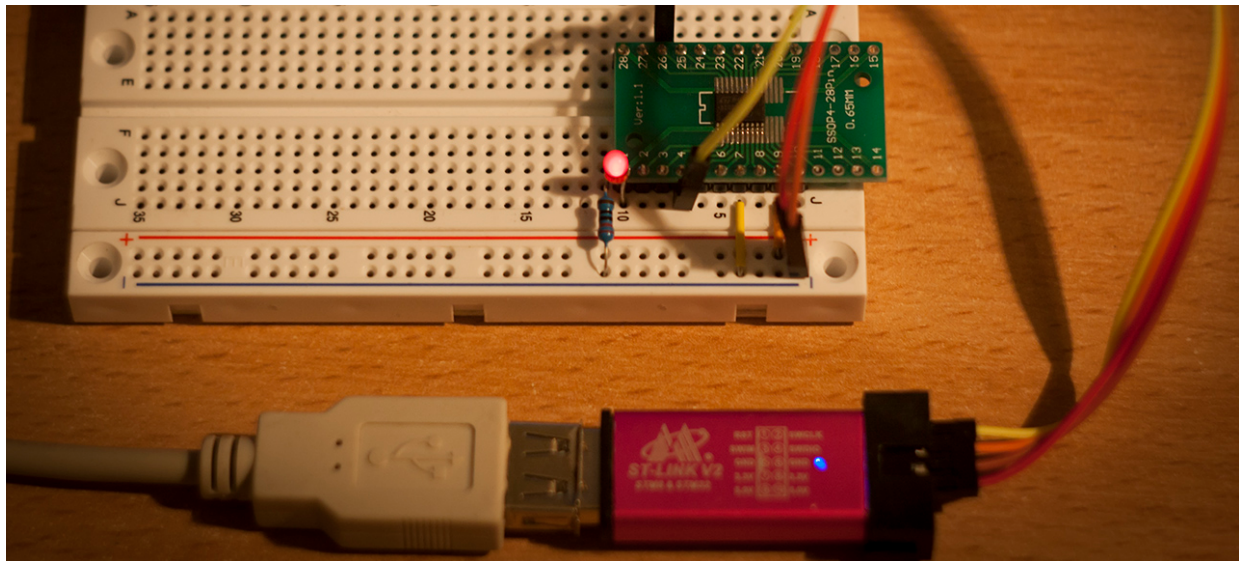
Save this in main.c and compile by running the following command:

```
1  sdcc -lstm8 -mstm8 --out-fmt-ihx --std-sdcc11 main.c
```

Now attach st-link and flash the microcontroller.

```
1  stm8flash -c stlinkv2 -p stm8s003f3 -w main.ihx
```





It's alive!

Congratulations! We've just written our first program from scratch.

Note: some of the STM8 pins are labeled with (T) in the datasheet. These pins are 'true' open-drain and can only pull to ground. You should be extra careful when working with open-drain pins, since there are no protection diodes. I managed to accidentally blow PB5 by using it as a normal GPIO, which took me hours to figure out when my I2C code wasn't working. One way of checking whether the pin is dead or not is by setting the multimeter in diode mode and measuring the voltage drop between the pin and ground - it should be roughly 0.7V in one direction.

Peripheral drivers

UART

After toggling some IO pins the first thing that you should get up and running on a new platform is UART. It makes debugging much easier. As always, we begin with register definitions.

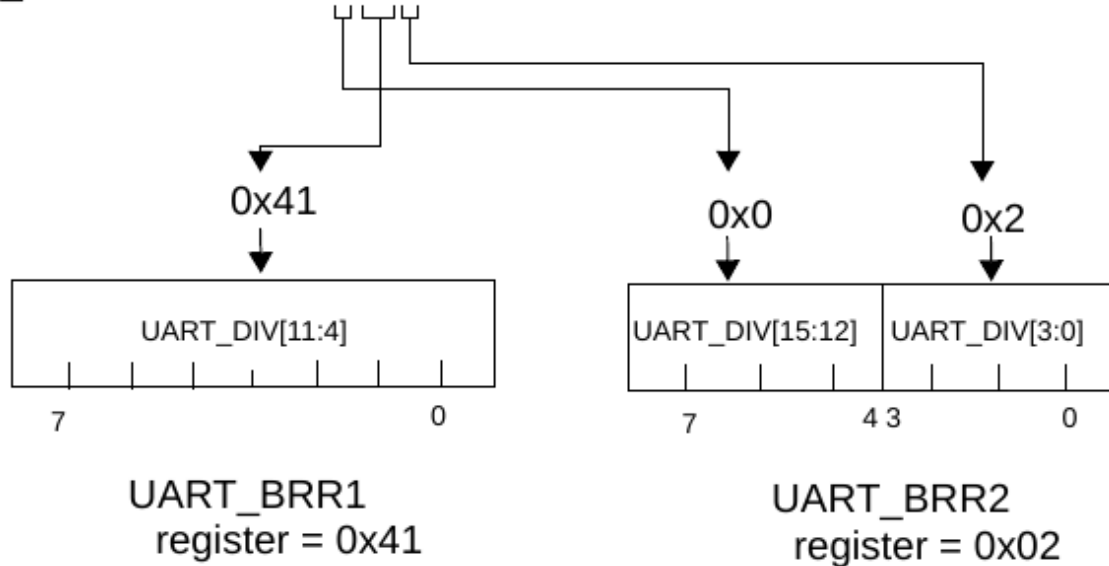
```
1  /* UART */
2  #define UART_SR      _SFR_(0x230)
3  #define UART_TXE     7
4  #define UART_TC      6
5  #define UART_RXNE    5
6
7  #define UART_DR       _SFR_(0x231)
8  #define UART_BRR1     _SFR_(0x232)
9  #define UART_BRR2     _SFR_(0x233)
10 #define UART_CR1      _SFR_(0x234)
11 #define UART_CR2      _SFR_(0x235)
12 #define UART_TEN      3
13 #define UART_REN      2
```

Usually, in order to initialize UART one has to calculate baud and write the resulting value into the corresponding HIGH and LOW registers. Let's see how this is done in STM8.

Example: To obtain 9600 baud with $f_{\text{MASTER}} = 10 \text{ MHz}$.

$\text{UART_DIV} = 10\,000\,000/9600$

$\text{UART_DIV} = 1042_{\text{d}} = 0x0412$



What were they thinking?!

So.. you get a 16-bit value and you write the first nibble [15:12] into BRR2[7:4], then you write bits [11:4] into BRR1 and finally you write the remaining bits [3:0] into BRR2[3:0]. Seriously, what were they thinking? Why couldn't ST just implement BRR_HIGH and BRR_LOW for the sake of it? All this bit-fiddling just seems unnecessarily complicated.

Anyway, let's move on to initialization. We'll stick with the default 8 data bits, 1 stop bit and no parity. Since our master clock is 2MHz, for baud = 9600 we have $\text{UART_DIV} = 2000000/9600 = 208$ (0xD0). According to the bizarre diagram above, we end up with BRR1 = 0x0D and BRR2 = 0x00. One thing to keep in mind is that BRR2 register must be written before BRR1. Finally, we turn on receiver and transmitter in Control Register 2. Read and write functions are pretty straight-forward: you read/write the Data Register and wait until the appropriate bit in Status Register is set.

```

1  /*
2   * PD5 -> TX
3   * PD6 -> RX
4   */
5  void uart_init() {
6      UART_BRR2 = 0x00;
7      UART_BRR1 = 0x0D;
8      UART_CR2 = (1 << UART_TEN) | (1 << UART_REN);
9  }
10
11 void uart_write(uint8_t data) {

```

```
12     UART_DR = data;
13     while (!(UART_SR & (1 << UART_TC)));
14 }
15
16 uint8_t uart_read() {
17     while (!(UART_SR & (1 << UART_RXNE)));
18     return UART_DR;
19 }
```

Redirecting `stdout` is easy with SDCC.

```
1 int putchar(int c) {
2     uart_write(c);
3     return 0;
4 }
```

Now we're all set and we can use `printf()` for debugging.

SPI

Next, we implement SPI master. SPI is quite an easy peripheral and is usually implemented as a simple shift-register in hardware. We need to define only 4 registers to start working with SPI.

```
1  /* SPI */
2  #define SPI_CR1    _SFR_(0x200)
3  #define SPE        6
4  #define BR0        3
5  #define MSTR        2
6  #define SPI_CR2    _SFR_(0x201)
7  #define SSM        1
8  #define SSI        0
9  #define SPI_SR      _SFR_(0x203)
10 #define BSY        7
11 #define TXE        1
12 #define RXNE        0
13 #define SPI_DR      _SFR_(0x204)
14
15 /* Chip select */
16 #define CS_PIN      4
```

Let's implement initialization and read/write functions. Reading from SPI is achieved by writing a dummy byte, so we'll hardcode `SPI_write(0xFF)` inside our `SPI_read()` function. Chip

select pin will be managed in software.

```
1  /*
2   * SPI pinout:
3   * SCK  -> PC5
4   * MOSI -> PC6
5   * MISO -> PC7
6   * CS   -> PC4
7   */
8  void SPI_init() {
9      /* Initialize CS pin */
10     PC_DDR |= (1 << CS_PIN);
11     PC_CR1 |= (1 << CS_PIN);
12     PC_ODR |= (1 << CS_PIN);
13
14     /* Initialize SPI master at 500kHz */
15     SPI_CR2 = (1 << SSM) | (1 << SSI);
16     SPI_CR1 = (1 << MSTR) | (1 << SPE) | (1 << BR0);
17 }
18
19 void SPI_write(uint8_t data) {
20     SPI_DR = data;
21     while (!(SPI_SR & (1 << TXE)));
22 }
23
24 uint8_t SPI_read() {
25     SPI_write(0xFF);
26     while (!(SPI_SR & (1 << RXNE)));
27     return SPI_DR;
28 }
29
30 void chip_select() {
31     PC_ODR &= ~(1 << CS_PIN);
32 }
33
34 void chip_deselect() {
35     PC_ODR |= (1 << CS_PIN);
36 }
```

To test our implementation I've written a simple loop that transmits some data.

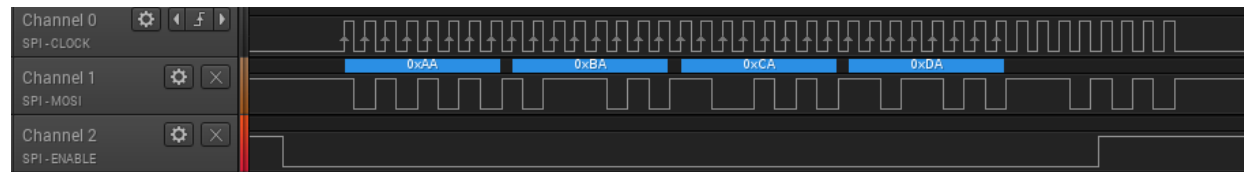
```
1  void main() {
2      SPI_init();
```

```

3     while (1) {
4         chip_select();
5         for (uint8_t i = 0xAA; i < 0xFA; i += 0x10)
6             SPI_write(i);
7         chip_deselect();
8     }
9 }

```

Let's hook up the logic analyzer and have a look.



SPI transmission

Hmm.. something is wrong. It seems that we release chip select too early and the last byte will not be received by a slave device. This can only occur if the SPI peripheral didn't have enough time to finish transmitting before we released CS pin.

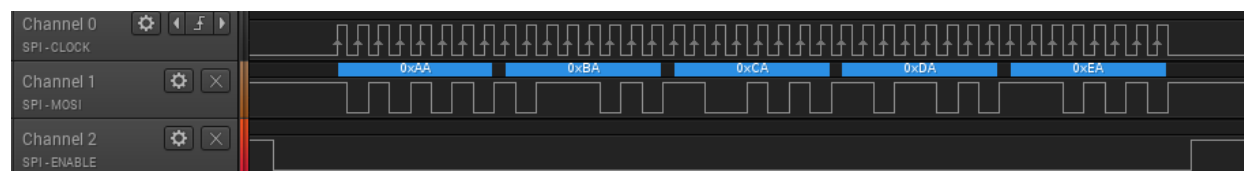
That wasn't supposed to happen - we are polling for `TXE` bit, aren't we? Well, the problem is that `TXE` only indicates that Tx buffer is empty. It doesn't tell us that all the bits were shifted out by the shift register. So in order to properly end the transmission we have to check for `BSY` flag, which tells us whether or not SPI has finished an operation. Let's modify our `chip_deselect()` function to take that into account.

```

1 void chip_deselect() {
2     while ((SPI_SR & (1 << BSY)));
3     PC_ODR |= (1 << CS_PIN);
4 }

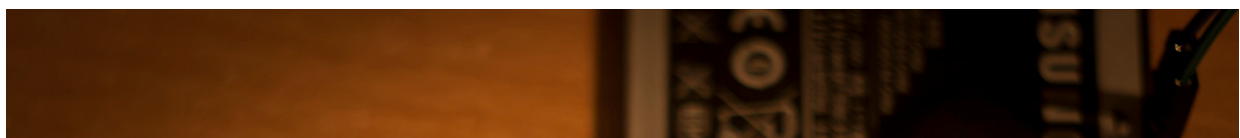
```

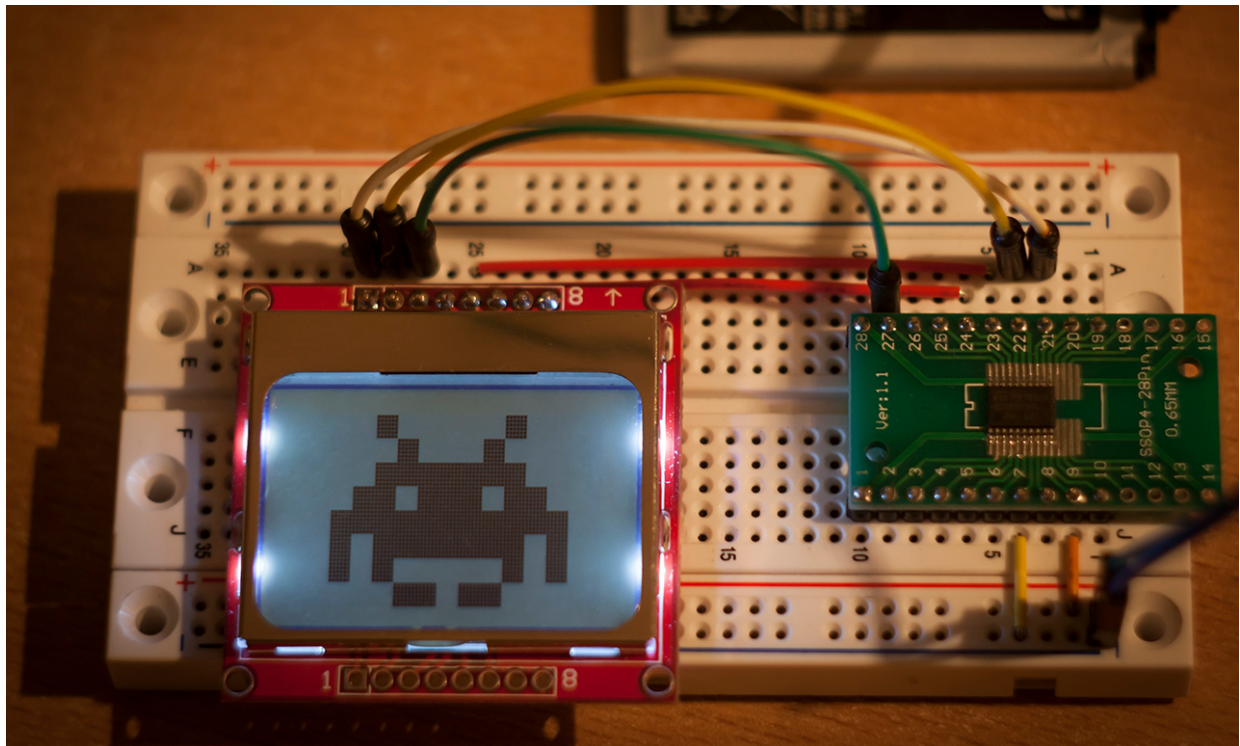
Final output.



SPI fixed

Our final test is the good old "Nokia 5110" LCD. Complete source is on [github](#).





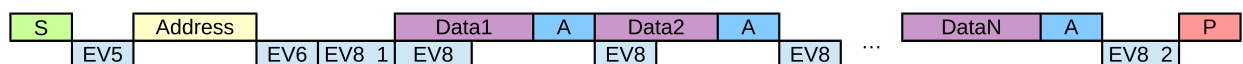
Nokia LCD

I2C

Now let's get onto something more serious. I2C usually requires a bit more work to get it up and running comparing to SPI and UART. I2C has a lot of associated registers, so I will no longer list them from this point. You can find a header with register definitions [here](#).

Let's take a look at what the reference manual says about receive and transmit operations.

7-bit master transmit



7-bit master receive



Legend

S Start
A ACK
NA NACK
P Stop

Events

EV5 SB=1, cleared by reading SR1 register followed by writing DR register with Address.
EV6 ADDR=1, cleared by reading SR1 register followed by reading SR3.
EV6_1 used for 1 byte reception only. Program ACK=0 and STOP=1 after clearing ADDR
EV7 RxNE=1, cleared by reading DR register
EV7_1 RxNE=1, cleared by reading DR register, program ACK=0 and STOP request
EV7_2 BTF=1, DataN-2 in DR and DataN-1 in shift register, program ACK=0, Read DataN-2 in DR
EV8_1 TXE=1, shift register empty, data register empty, write DR register
EV8 TXE=1, shift register not empty, data register empty, cleared by writing DR register
EV8_2 TXE=1, BTF = 1, Program STOP request. TXE and BTF are cleared by HW by stop condition
EV9 ADD10=1, cleared by reading SR1 register followed by writing DR register

I2C transmit and receive modes

That does seem quite complicated: a lot of events are generated during communication. However, we don't have to explicitly take care of every single event in order to have a working communication - some of the events are automatically cleared by hardware and some may just be ignored and left unattended. We'll go with the easiest implementation.

We start by implementing initialization and IO functions. We also need dedicated functions to generate start and stop conditions.

```
1  /*
2   * I2C pinout:
3   * SCL -> PB4
4   * SDA -> PB5
5   */
6  void i2c_init() {
7      I2C_FREQR = (1 << I2C_FREQR_FREQ1);
8      I2C_CCRL = 0x0A; // 100kHz
9      I2C_OARH = (1 << I2C_OARH_ADDMODE); // 7-bit addressing
10     I2C_CR1 = (1 << I2C_CR1_PE);
11 }
12
13 void i2c_start() {
14     I2C_CR2 |= (1 << I2C_CR2_START);
15     while (!(I2C_SR1 & (1 << I2C_SR1_SB)));
16 }
17
18 void i2c_stop() {
19     I2C_CR2 |= (1 << I2C_CR2_STOP);
20     while (I2C_SR3 & (1 << I2C_SR3_MSL));
21 }
22
23 void i2c_write(uint8_t data) {
24     I2C_DR = data;
25     while (!(I2C_SR1 & (1 << I2C_SR1_TXE)));
26 }
27
28 uint8_t i2c_read(uint8_t ack) {
29     if (ack)
30         I2C_CR2 |= (1 << I2C_CR2_ACK);
31     else
32         I2C_CR2 &= ~(1 << I2C_CR2_ACK);
33     while (!(I2C_SR1 & (1 << I2C_SR1_RXNE)));
34     return I2C_DR;
35 }
```

According to the reference manual, writing slave address is a special case so we can't simply use `i2c_write()` to do that. We need a dedicated function for this purpose.

```
1  void i2c_write_addr(uint8_t addr) {
2      I2C_DR = addr;
3      while (!(I2C_SR1 & (1 << I2C_SR1_ADDR)));
4      (void) I2C_SR3; // clear EV6
```

```
5     I2C_CR2 |= (1 << I2C_CR2_ACK);  
6 }
```

Reference manual says we are supposed to to handle EV6 event after writing slave address: “EV6: ADDR=1, cleared by reading SR1 register followed by reading SR3”. After polling for ADDR bit we simply read SR3 register. I’m not sure why this is required, probably to check for BUS_BUSY, but that seemed a bit pointless so we cheated a little.

Now, let’s test our library with an HMC5883L magnetometer. First we define R/W flags and some magnetometer related stuff:

```
1  #define I2C_READ          0x01  
2  #define I2C_WRITE         0x00  
3  
4  #define HMC5883_ADDR      (0x1E << 1)  
5  #define HMC5883_CR_A      0x00  
6  #define HMC5883_CR_B      0x01  
7  #define HMC5883_MODE       0x02  
8  #define HMC5883_DATA_OUT   0x03  
9  #define HMC5883_ID_REG_A   0x0A
```

We’ll implement a simple function that reads the device Id and sends it over UART.

```
1  void hmc5883_get_id(uint8_t *id) {  
2      /* Tell device we want to read ID_REG_A */  
3      i2c_start();  
4      i2c_write_addr(HMC5883_ADDR + I2C_WRITE);  
5      i2c_write(HMC5883_ID_REG_A);  
6      i2c_stop();  
7  
8      /* Read ID bytes */  
9      i2c_start();  
10     i2c_write_addr(HMC5883_ADDR + I2C_READ);  
11     id[0] = i2c_read(1);  
12     id[1] = i2c_read(1);  
13     id[2] = i2c_read(0);  
14     i2c_stop();  
15 }  
16  
17 int main() {  
18     uint8_t id[3];  
19     uart_init();
```

```

20     i2c_init();
21
22     while (1) {
23         hmc5883_get_id(id);
24         printf("Device ID: %c%c%c\n", id[0], id[1], id[2]);
25         delay_ms(250);
26     }
27 }

```

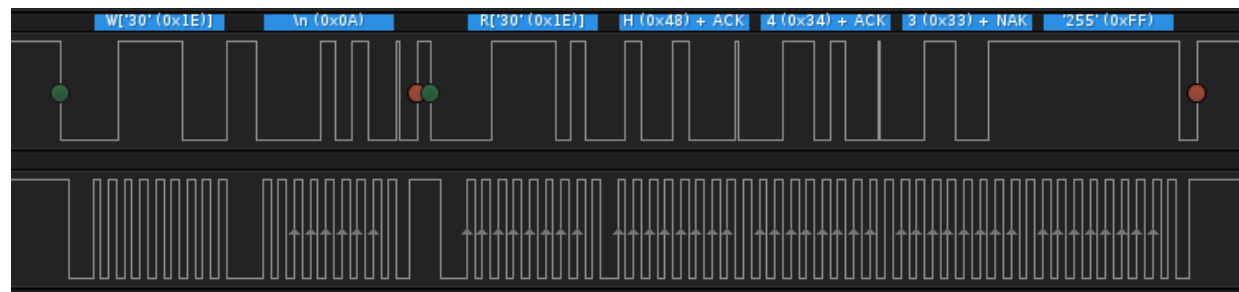
Output:

```

1  Device ID: H43

```

All seems to work fine, but let's take a look at the logic analyzer just to make sure.



I2C receiver (broken)

Hmm.. we do receive correct bytes, but what's the deal with that 0xFF received right after the NACK? It seems that something is wrong with our code. Time to RTFM.

The Proper Way

So the first problem is how we generate STOP condition. According to the documentation, we are supposed to generate STOP *before* reading the last byte. I changed the code but it didn't fix the problem. The real problem was that I was porting the magnetometer driver which I wrote for a different microcontroller, so I *expected* the I2C peripheral to work in a certain way. Well, I was wrong.

The `i2c_read()` function is supposed to receive only 1 byte of data. It turns out there are 3 different scenarios for $N=1$, $N=2$ and $N>2$, where N is the number of received bytes. We can't simply use the function for $N=1$ to read more than a single byte. That means we need separate functions to handle each case! I wonder how many logic gates were dedicated to implement I2C peripheral on this MCU... (Note: I2C implementation on STM32F1xx series is actually identical to STM8.)

Looking at the reference manual I figured that we could possibly combine $N=2$ and $N>2$ cases and handle them with a single function. Below are proper implementations of I2C receive functions.

```

1  uint8_t i2c_read() {
2      I2C_CR2 &= ~(1 << I2C_CR2_ACK);
3      i2c_stop();
4      while (!(I2C_SR1 & (1 << I2C_SR1_RXNE)));
5      return I2C_DR;
6  }
7
8  void i2c_read_buf(uint8_t *buf, int len) {
9      while (len-- > 1) {
10         I2C_CR2 |= (1 << I2C_CR2_ACK);
11         while (!(I2C_SR1 & (1 << I2C_SR1_RXNE)));
12         *(buf++) = I2C_DR;
13     }
14     *buf = i2c_read();
15 }

```

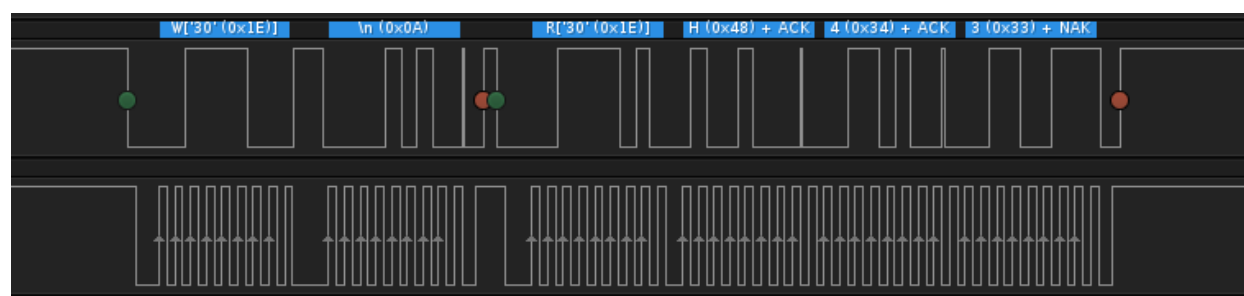
Now let's update our code for reading device Id.

```

1  void hmc5883_get_id(uint8_t *id) {
2      /* Tell device we want to read ID_REG_A */
3      i2c_start();
4      i2c_write_addr(HMC5883_ADDR + I2C_WRITE);
5      i2c_write(HMC5883_ID_REG_A);
6      i2c_stop();
7
8      /* Read ID bytes */
9      i2c_start();
10     i2c_write_addr(HMC5883_ADDR + I2C_READ);
11     i2c_read_buf(id, 3);
12 }

```

Note that our `i2c_read_buf()` function generates STOP so we no longer have to call `i2c_stop()` manually. Let's take a look at the logic analyzer now.



I2C fixed

Great, no 0xFF at the end! Now we're ready to move onto something different.

ADC

Nothing exciting about the ADC on STM8: 10-bit resolution, single and continuous conversion modes, configurable prescaler.. all the usual boring stuff. There is also a data buffer that can hold a number of ADC samples, which is rather convenient.

The default `printf()` implementation provided by SDCC does not support floats. To enable floating point output, `printf_large.c` needs to be recompiled with `-DUSE_FLOATS=1` option. For this example we are going to cheat and print the results in millivolts instead. Without further ado, let's write some code for single ADC conversion.

```
1  #define V_REF 3.3
2
3  uint16_t ADC_read() {
4      uint8_t adcH, adcL;
5      ADC1_CR1 |= (1 << ADC1_CR1_ADON);
6      while (!(ADC1_CSR & (1 << ADC1_CSR_EOC)));
7      adcL = ADC1_DRL;
8      adcH = ADC1_DRH;
9      ADC1_CSR &= ~(1 << ADC1_CSR_EOC); // Clear EOC flag
10     return (adcL | (adcH << 8));
11 }
12
13 void ADC_init() {
14     /* Configure ADC channel 4 (PD3) */
15     ADC1_CSR |= (1 << 2);
16     /* Right-align data */
17     ADC1_CR2 |= (1 << ADC1_CR2_ALIGN);
18     /* Wake ADC from power down */
19     ADC1_CR1 |= 1 << ADC1_CR1_ADON;
20 }
21
22 void main() {
23     ADC_init();
24     uart_init();
25
26     while (1) {
27         uint16_t val = ADC_read();
28         float voltage = (V_REF / 1024.0) * val * 1000;
29         printf("Channel4: %d mV\n", (uint16_t) voltage);
30         delay_ms(250);
31     }
32 }
```

Pretty straight forward. Note that `EOC` flag has to be manually cleared by software.

A few things that should be taken into account when working with ADC:

- The order in which `DRL` and `DRH` registers are accessed depends on data alignment.
- ADC has no internal voltage reference. STM8S003 does not have an external Vref pin, so it is tied to Vcc internally, which means that your supply voltage has to be spot-on for any serious measurements.
- Data buffer registers have no internal locking. ST provides an assembly snippet in the datasheet for reading buffer registers.

Timers and interrupts

You can't get far without using timers and interrupts, which is what this last section will cover.

STM8S003 has 16-bit 'advanced control' as well as 8-bit general-purpose timers. TIM1 is a really complicated peripheral with 32 dedicated registers, and covering it's functionality would probably require a few extra articles. For this article, we'll use TIM4 which is good enough for basic applications.

There isn't much to tweak inside TIM4: it contains an 8-bit auto-reload up counter, 3-bit prescaler and an option to generate interrupt on counter overflow.

The prescaler divides counter clock frequency by a power of 2 from 1 to 128 depending on `PSCR` registers:

$$prescaler = 2^{PSCR[2:0]}$$

In this example we are going to toggle a pin each time the counter matches value in the `ARR` register. The frequency of the waveform generated by our IO pin is calculated as follows:

$$f_{io} = \frac{f_{clk}}{2 \cdot prescaler \cdot (1 + ARR)}$$

To achieve a frequency of 100Hz `ARR` has to be set to 77, given that our clock frequency is 2MHz. We need to enable Update Interrupt for `TIM4`, but before that interrupts must be enabled globally by executing `rim` instruction.

```
1  int main() {
2      /* Enable interrupts */
3      __asm__("rim");
4
5      /* Set PD3 as output */
6      PD_DDR |= (1 << OUTPUT_PIN);
7      PD_CR1 |= (1 << OUTPUT_PIN);
8
9      /* Prescaler = 128 */
10     TIM4_PSCR = 0b00000111;
11
12     /* Period = 5ms */
13     TIM4_ARR = 77;
14
15     TIM4_IER |= (1 << TIM4_IER_UIE); // Enable Update Interrupt
16     TIM4_CR1 |= (1 << TIM4_CR1_CEN); // Enable TIM4
17
18     while (1) {
19         /* Loop forever */
```

```

20     }
21 }

```

Now, when I said that we're going to implement everything from scratch, I wasn't completely honest. We're still using some start-up code which initializes the stack and interrupt vector table. If you look at the listing you can see that SDCC has generated the interrupt table for us:

```

1  000000 82v00u00u00          37      int s_GSINIT ;reset
2  000004 82 00 00 00          38      int 0x0000 ;trap
3  000008 82 00 00 00          39      int 0x0000 ;int0
4  ...
5  00007C 82 00 00 00          68      int 0x0000 ;int29

```

Registering an interrupt handler is easy with SDCC: there is a special attribute `_interrupt()` which takes interrupt number as a parameter. Section 7 ('Interrupt vector mapping') of the datasheet describes which IRQ number corresponds to which peripheral. For TIM4 it is 23. Our interrupt handler will look like this:

```

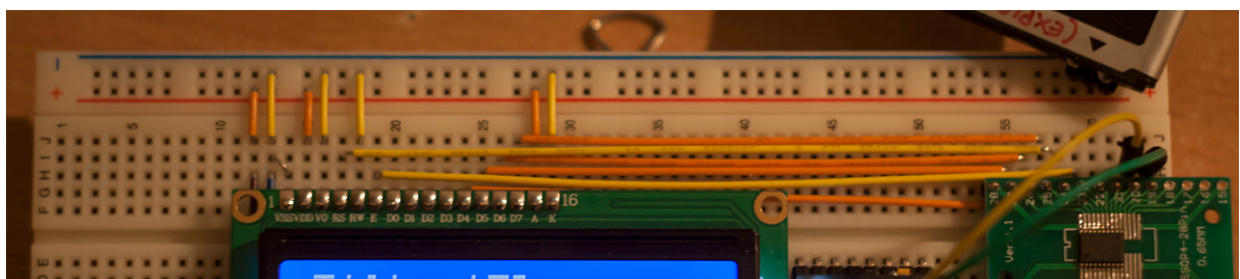
1  #define TIM4_ISR 23
2
3  void timer_isr(void) __interrupt(TIM4_ISR) {
4      PD_ODR ^= (1 << OUTPUT_PIN);
5      TIM4_SR &= ~(1 << UIF);
6  }

```

Putting it all together

We have enough building blocks - now it's time to put them together into some 'real-world' application. For this demo I picked up [MMA8452](#) 3-axis I2C accelerometer and a standard [HD44780](#) 1602 LCD, which is extremely popular among electronics enthusiasts for some reason.

The demo application will calculate inclination angle based on accelerometer readings and output it to the LCD. Calculating inclination angle will require some trigonometry and floating point arithmetic, which will consume a good amount of resources. Despite the floating point operations being quite slow, STM8 managed this task decently.





Demo

You might have noticed the lack of contrast adjustment potentiometer. The LCD module that I'm using is rated for 5V, however my setup uses 3.3V supply. I couldn't be bothered with a separate supply for the display, so I cheated: the LCD is initialized in 1-line mode, which results in 1/8 duty cycle, and `Vo` pin is tied to ground.

Conclusion

STM8 is nice and cheap, but it is really hard to justify using this microcontroller, especially given the fact that price difference between STM8 and low-end Cortex-M0 devices like STM32F03 is negligible. The biggest downside for me was lack of GCC support. Despite SDCC being a reasonably good compiler, it does not fully support C99 and C11 standards, which means that I have to refactor most of my existing code to make it compatible. Code optimization isn't great either, which is a shame, since most STM8 microcontrollers don't have a lot of flash to spare.

As always, code is available on [github](#).

#bare-metal #sdcc #stm8

💬 Comments ➦ Share

NEWER POSTS

[Bare metal programming: STM8 \(Part 2\)](#)

OLDER POSTS

[Installing Black Magic via ST-Link bootloader](#)

26 Comments

Lujji's blog

 Disqus' Privacy Policy Login ▾ Recommend 5 Tweet Share

Sort by Best ▾



Join the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS Name **Miguel Angel Yepes Valencia** • 3 years ago

Very nice post Lujji!

I am new with stm8 but I have programmed PIC Microcontrollers.
You could tell me how I can configure the SPI module to work in the master mode, using only the MOSI pin, leaving the MISO pin as I / O for other tasks of the application and also handle the selection of the slave by software, thus also leaving the NSS pin as I / O?

11  |  • Reply • Share >**Lujji**  Miguel Angel Yepes Valencia • 3 years ago

Take a look at SPI_CR2 register in the reference manual: you'll need to enable BDM and BDOE bits for single line transmit-only mode. Setting SSM and SSI bits allows you to manage chip select in software, although it seems that you can't use NSS as an IO since the value of the SSI bit will be forced onto this pin.

1  |  • Reply • Share >**galagonya.gergo** • 3 years ago

Very nicely, Good job!

I like this contorller, use audio digital pot and other audio little work.
very thorough post, i use it as well.
I'am port stm8flash util to windows and i use it.

1  |  • Reply • Share >**golfman484** • 3 years ago

Thanks man - you gave me some flashbacks to the 80's when I used to build lots of fun things with the Motorola 68705 - EEPROM programmable 68x05 series single chip microcontroller. It was so much fun making devices and coding them to do actual real world things - much more fun than writing software in the finance industry that I do these days - and much fewer idiots to deal with back then too! ;)

1  |  • Reply • Share >**Joe** • 5 months ago • edited

This appears to be old now, but interesting. Do you have a link to the chip and the

^ | v • Reply • Share ›



sdev • 7 months ago

@lujji @I am trying to use I2C example with RTC DS1307. but when controller get stuck somewhere in i2C.

Do you have any example ready for DS1307.

^ | v • Reply • Share ›



Shiva Kumar R • a year ago

Hi

Very good post. I really started understanding the STM8 chips after I read this. One heads up. The stm8s.h file needs a minor correction. The root address definition for TIM2 in the file is in decimal instead of hex. I found out after 3 days of beating around forums, mailing lists and information exchanges (Thank you Richard Hodges. You saved me from a lot of heartburn, frustration and depression!) Once again, Lujji, thank you for this lovely, eye opening post.

^ | v • Reply • Share ›



DaniNedo • 2 years ago • edited

Amazing article!

I was analyzing the SDCC command and trying to understand what the different flags do. I found information about `-mstm8`, `--out-fmt-ihx` and `--std-sdcc11` in the SDCC user guide, but no clue about the `-lstm8`. I've tried just executing `sdcc -lstm8 main.c` and I see that a `.mem` file is generated (a quite useful file actually) however I received some warnings with the code 88. This `.mem` file is not generated when executing the complete command presented in this post.

Can you explain what is `-lstm8` used for? Thank you in advance!

^ | v • Reply • Share ›



ryanplusplus → DaniNedo • 2 years ago

It tells the compiler to link `stm8.lib` (that is, standard library support for the STM8). I believe that it's implied when you use `-mstm8` so isn't technically necessary.

^ | v • Reply • Share ›



DaniNedo → ryanplusplus • 2 years ago • edited

Thanks for replying! Indeed when I execute `sdcc -stm8 --out-fmt-ihx --std-sdcc11 main.c` the files are generated just fine. It seems that you are right about the implicit call of `-lstm8` because adding `--verbose` shows that the linker calls `main.lk` and in that file there's

^ | v • Reply • Share ›



DaniNedo → ryanplusplus • 2 years ago • edited

I'm also new to SDCC so no worries. I don't think there's much more going on with the .mem file, just that the info is shown more clearly, I'm a bit lost with the .map file. Can you explain how do you determine the RAM and ROM usage from .map? My .mem looks like [this](#) and the .map like [this](#). Notice that they correspond to different targets.

^ | v • Reply • Share ›



ryanplusplus → DaniNedo • 2 years ago

I took a look at your map file and can't explain how to get to the ROM number shown (213), but in general the CODE and HOME sections hold the actual instructions from your code, GSINIT contains (I'm pretty sure) the startup/init code, DATA contains RAM data, CONST contains ROM data, INITIALIZED contains initialized static variables (so this occupies both RAM and ROM). I believe SSEG is the stack segment and reflects the amount of reserved space for stack, but I'm not 100% sure (it shows as 1 in my project...). I don't know what DSEG or GSFINAL are.

^ | v • Reply • Share ›



DaniNedo → ryanplusplus • 2 years ago

Cool, thanks a lot for the help!

^ | v • Reply • Share ›



ByteMaster • 3 years ago

Great work Lujji!

I've compiled main.c using following command

```
sdcc -lstm8 -mstm8 --out-fmt-s19 --std-sdcc11 main.c
```

However, generated .s19 is out of range when loaded in STVP.

Is there any other steps need to be followed to make it Final .s19?

P.S. I'm not using Stm8Flash

^ | v • Reply • Share ›

about ESP8266 - had a bunch of them all over the place and not a single project in mind. I do intend to make a follow-up article on STM8, so if you're interested in some particular topic, let me know.

1 ^ | v • Reply • Share ›



Анна Макарова → lujji • 4 years ago • edited

I wonder where my huge answer with ideas has gone. Anyway, let's try again in short. May be some interests would be in common:

- Clocking systems in general. How one does bare metal over\underclocking particular interfaces or/and the whole system keeping it stable;
- Is there any inherited hardware (ram\u005Crom\u005Cinterfaces) from more expensive chips that can be activated\u005Cused in cheapest chips;
- Non-trivial usage of common serial interfaces or/and GPIO bit-banging (VGA,Ethernet,vUSB etc.), may be with assembly in SDCC and problems with instruction clock timings caused by internal CPU optimizations ;
- Why do people blame STM's hardware being worse than competitors and what are the workarounds? (I2C,SMBus,IMBus is blamed to hang by itself, SPI is said to have less modes than others do etc.);
- Is there unique(including hidden) STM8 hardware\u005Cmodes and ways to use (UID, touch, quadrature inputs\u005Coutputs, beepers, CAN, LIN etc.) available?
- Laying out memory blocks explicitly, forcing compiler to place things where we want to. Playing with memories, differences between EEPROM and flash in operation, working around simultaneous write and then read to EEPROM and Flash. Code execution from ram (if possible).
- Internal clock stability and hw\u005Csoftware compensations (trimming?)
- Everything on sleep modes and ways to halt CPU core for some cycles without wasting power on NOPs and jumps (infinite loops) and

me, but mostly I'm doing analog, jelly-bean logic, SOC or F-C-driven projects only, no contemporary MCUs involved. Hope it will change)

The main problem was the lack of time to (constantly re-)learn third-party or vendor's (= low quality now days) libraries. It looks like they are poorly optimized, full of errors, backward-compatibility breakage, non-uniform and non-compatible with datasheets terminology. The very presence of HALs, SPLs, *duinos etc. was very misleading and makes people (including me) think that bare HW is very obscure and unreachable to mere human. Your blog plays important enlightenment role. Even datasheets are full of mistakes and ambiguities and no one corrects them because people just do not dare to get the responsibility for the whole system themselves i.e. do not use direct HW approach. The less we work with metal, the less quality HW and documentation we will get in the future.

Considering future topics for novice like me it's very interesting to dig out undocumented or poorly documented features either good or bad ones. Test-out, activate and use "hidden" memory or "non-documented" hardware "inherited" from more expensive models with the same silicon chip, for example. Practical bare metal sleep modes including the most economic deep sleep or just idling without wasting power to infinite loop while HW is doing or waiting for something (besides polling there are a lot of "-nop"-filled loops that look like total power waste. Clocking modules management (may be even overclocking, underclocking).

Basically the more real-life "underwater rocks" you mention and work around, the better it would be. It's very interesting to see step-by-step

seems plausible. when you buy parts directly from the manufacturer in bulk quantities, the prices tend to be way lower than what you see on digikey/mouser/etc. Quite often you can buy MCUs from places like aliexpress for ridiculous prices - in my case I had no problems with these parts.

Although I used various STM8 MCUs while writing this article, [STM8S003F3](#) definitely has 3 timers: TIM1, TIM2 and TIM4.

^ | v • Reply • Share ›



kali prasad yadav • 4 years ago • edited

Excellent post Lujji!

It's really hard to find good info on STM8s with SDCC. Hoping more blog posts on stm8 may be some display modules like SSD1306 OLED, ST7735 LCD etc..

^ | v • Reply • Share ›



lujji Mod → kali prasad yadav • 4 years ago

Thanks, I might do some more posts on STM8 in future. The displays you mentioned are SPI/I2C, so you can easily port existing libraries by replacing IO functions.

^ | v • Reply • Share ›

ARCHIVES

[August 2017](#)
[July 2017](#)
[April 2017](#)
[March 2017](#)
[February 2017](#)
[January 2017](#)
[October 2016](#)
[September 2016](#)

RECENT POSTS

[Serial bootloader for STM8](#)
[Mixing C and assembly on STM8](#)
[Executing code from RAM on STM8](#)
[Bare metal programming: STM8 \(Part 2\)](#)
[Bare metal programming: STM8](#)

© 2017 Iujji
Iujji at protonmail com